

SQL Advanced Join Techniques

2 · Advanced Joins — Complete Reference Guide (Banking Project)

Table of Contents

1. LEFT ANTI JOIN
2. RIGHT ANTI JOIN
3. FULL ANTI JOIN
4. CROSS JOIN
5. How to Choose the Right Join
6. How to Join Multiple Tables

1. LEFT ANTI JOIN

→ **Task 1: Find every customer who has never opened a single account, so the onboarding team has a clean target list to follow up with.**

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    c.customer_segment, c.registration_date
FROM Customers c
LEFT JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE a.account_id IS NULL;
```

→ **Task 2: Find every account that has never had a single transaction posted against it, for the zero-activity account list.**

```
SELECT
    a.account_id, a.account_type, a.account_status, a.open_date
FROM Accounts a
LEFT JOIN Transactions t
    ON a.account_id = t.account_id
WHERE t.transaction_id IS NULL;
```

→ **Task 3: Find customers who own at least one account but have never made a single transaction on any of their accounts -- the least-engaged customer list, built with a LEFT ANTI JOIN across all three tables.**

```
SELECT DISTINCT
    c.customer_id, c.first_name, c.last_name, c.customer_segment
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
LEFT JOIN Transactions t
    ON a.account_id = t.account_id
WHERE t.transaction_id IS NULL;
```

2. RIGHT ANTI JOIN

→ **Task 4: Using a RIGHT ANTI JOIN (RIGHT JOIN + NULL check) from Customers to Accounts, check for any account that does not belong to a valid customer record -- a data-integrity check. Because customer_id is a foreign key in this database, this correctly returns zero rows -- which is exactly what a clean integrity check should show.**

```
SELECT
    a.account_id, a.account_type, a.account_balance
FROM Customers c
RIGHT JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE c.customer_id IS NULL;
```

→ **Task 5: Using a RIGHT ANTI JOIN from Accounts to Transactions, check for any transaction that does not belong to a valid account -- again a data-integrity check that should return zero rows in a healthy database.**

```
SELECT
    t.transaction_id, t.transaction_type, t.transaction_amount
FROM Accounts a
RIGHT JOIN Transactions t
    ON a.account_id = t.account_id
WHERE a.account_id IS NULL;
```

→ **Task 6: Wrap the same RIGHT ANTI JOIN pattern in a COUNT so it can run daily as an automated orphan-record monitor for the data-quality team, rather than a one-off manual check.**

```
SELECT
    COUNT(*) AS orphan_account_count
FROM Customers c
RIGHT JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE c.customer_id IS NULL;
```

3. FULL ANTI JOIN

→ **Task 7: Combine customers without accounts and accounts without a matching customer into a single unmatched-only report using a FULL ANTI JOIN between Customers and Accounts.**

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    a.account_id, a.account_type
FROM Customers c
FULL JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE c.customer_id IS NULL
    OR a.account_id IS NULL;
```

→ **Task 8: Combine accounts without transactions and transactions without a matching account into a single unmatched-only report using a FULL ANTI JOIN between Accounts and Transactions.**

```
SELECT
    a.account_id, a.account_type,
    t.transaction_id, t.transaction_type
FROM Accounts a
FULL JOIN Transactions t
    ON a.account_id = t.account_id
WHERE a.account_id IS NULL
    OR t.transaction_id IS NULL;
```

→ **Task 9: Extend the FULL ANTI JOIN with a CASE expression that labels each unmatched row as 'Account Without Customer' or 'Customer Without Account', for the reconciliation team's exceptions report.**

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    a.account_id, a.account_type,
    CASE
        WHEN c.customer_id IS NULL THEN 'Account Without Customer'
        ELSE 'Customer Without Account'
    END AS exception_type
FROM Customers c
FULL JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE c.customer_id IS NULL
    OR a.account_id IS NULL;
```

4. CROSS JOIN

→ **Task 10: Build a product-recommendation matrix by cross joining every distinct customer segment with every distinct account type, so the marketing team has a template of every segment-product combination to plan campaigns around.**

```
SELECT
    seg.customer_segment, typ.account_type
FROM (SELECT DISTINCT customer_segment FROM Customers) seg
CROSS JOIN (SELECT DISTINCT account_type FROM Accounts) typ
ORDER BY seg.customer_segment, typ.account_type;
```

→ **Task 11: Build a branch-by-product matrix by cross joining every distinct branch with every distinct account type, so operations can plan which product literature each branch should stock.**

```
SELECT
    b.branch_name, typ.account_type
FROM (SELECT DISTINCT branch_name FROM Accounts) b
CROSS JOIN (SELECT DISTINCT account_type FROM Accounts) typ
ORDER BY b.branch_name, typ.account_type;
```

→ **Task 12: Cross join every Premium-segment customer with every distinct transaction channel, to build a channel-availability checklist template the digital banking team can populate with adoption flags.**

```
SELECT
    c.customer_id, c.first_name, c.last_name, ch.transaction_channel
FROM (
    SELECT customer_id, first_name, last_name
    FROM Customers
    WHERE customer_segment = 'Premium'
) c
CROSS JOIN (SELECT DISTINCT transaction_channel FROM Transactions) ch
ORDER BY c.customer_id, ch.transaction_channel;
```

5. How to Choose the Right Join

→ **Task 13: Business ask: 'Show me only customers with active accounts and their balances.'** Since we only want customers that genuinely have a matching active account, **INNER JOIN** is the right choice -- it drops anything unmatched, which is exactly what this request wants.

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    a.account_id, a.account_balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE a.account_status = 'Active';
```

→ **Task 14: Business ask: 'I need a complete customer contact list for a marketing email blast, regardless of whether they have an account.'** Here we must keep every customer row even without a match, so **LEFT JOIN** is the right choice, not **INNER JOIN**.

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.city,
    a.account_id
FROM Customers c
LEFT JOIN Accounts a
    ON c.customer_id = a.customer_id;
```

→ **Task 15: Business ask: 'I need every transaction matched with its account and customer details, and I must never lose a transaction row even if its account or customer link were missing.'** Starting from Transactions and using **LEFT JOIN** at each step protects every transaction row -- this is the correct choice over **INNER JOIN**, which would silently drop transactions with a broken link.

```
SELECT
    t.transaction_id, t.transaction_type, t.transaction_amount,
    a.account_id, a.account_type, c.first_name, c.last_name
FROM Transactions t
LEFT JOIN Accounts a
    ON t.account_id = a.account_id
LEFT JOIN Customers c
    ON a.customer_id = c.customer_id;
```

6. How to Join Multiple Tables

→ **Task 16: Join all three tables together -- Customers, Accounts, and Transactions -- to produce a single 360-degree view of each transaction with full customer and account context.**

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.customer_segment,
    a.account_id, a.account_type, a.account_status,
    t.transaction_id, t.transaction_type,
    t.transaction_amount, t.transaction_date
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
INNER JOIN Transactions t
    ON a.account_id = t.account_id;
```

→ **Task 17: Join all three tables using LEFT JOIN throughout so that customers with no accounts, and accounts with no transactions, all still appear in one complete relationship audit, ordered by customer.**

```
SELECT
    c.customer_id, c.first_name, c.last_name,
    a.account_id, a.account_type,
    t.transaction_id, t.transaction_amount
FROM Customers c
LEFT JOIN Accounts a
    ON c.customer_id = a.customer_id
LEFT JOIN Transactions t
    ON a.account_id = t.account_id
ORDER BY c.customer_id;
```

→ **Task 18: Join all three tables and aggregate to calculate the total successful transaction amount per customer, per account type -- feeding directly into the segmentation team's account-profitability model.**

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.customer_segment,
    a.account_type,
    SUM(t.transaction_amount) AS total_successful_amount
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
INNER JOIN Transactions t
    ON a.account_id = t.account_id
WHERE t.transaction_status = 'Success'
GROUP BY c.customer_id, c.first_name, c.last_name,
    c.customer_segment, a.account_type;
```